

Haller + Erne GmbH

heOGS – Live video integration **Application Note**

HEI-21-0611
Version R01

Version history

Version	History	Author	Date
R01	First release	he	2021-06-11

About this document

This document contains information about how to integrate a live video feed into the heOGS operator guidance software. In addition to showing the general setup, it also describes a solution to integrate live, low latency CCTV camera streams, which are typically not viewable (or only with high latency) through a standard web browser.

The solution describes how to setup two public domain, free and open source software applications to create a background service, which converts the live camera stream in real time from the native RTSP format into the webRTC format used in modern web browsers.

Table of Contents

1	Overview	4
2	Installation	5
2.1.1	Download software	5
2.1.2	Unpack software	5
2.2	Quick test	5
2.3	Service installation and configuration	6
2.4	Test	9
2.5	Tweaking	9
3	OGS integration	11
3.1	Workflow configuration	11
3.2	OGS Runtime	12

1 Overview

For some processes or configurations, live video must be shown during a process step or the operator should capture some photo where he needs to ensure the proper view is visible. In both situations, OGS must show a live video feed, with preferably low latency and a high frame rate to get a real-time view on the screen.

As OGS provides an integrated web browser view, which can be assigned to a process step/task, it is in general easy to create a web page which embeds the camera live video stream and show it for a given process step/task.

However, there are a few technical issues:

- Typical CCTV cameras only provide rtsp streams, which are not understood by web browsers
- Other common video formats supported by cameras and browsers (like motion JPEG) have high latency or poor frame rates (e. g. only one picture per second), so don't really provide a "realtime" view
- Hosting a camera feed through a html page stored on the local filesystem imposes "cross origin" security restrictions and might be blocked by the Microsoft Edge WebView2 browser used inside OGS.

The solution to fix all these issues is presented in this application note. It uses two open source, public domain software components to create a background service, which converts the cameras rtsp stream into a webRTC stream, which then gets rendered natively in the Microsoft Edge WebView inside OGS.

The software components used are:

- webRTC-streamer (see <https://github.com/mpromonet/webRTC-streamer>). This software converts rtsp streams into a webRTC stream for use in browsers. The github "Releases" page of the project provides prebuild binaries for Windows 10/64.
- nssm (see <https://nssm.cc/>), the "no sucking service manager" is used to install the webRTC-streamer as a windows service to ensure the webRTC-streamer application is available regardless who is logged on to the system.

The following chapters describe on how to download, install and configure these software components, as well as how to use them with a Axis CCTV camera and integrate a live view into an OGS process flow.

2 Installation

To install the software components (this is a one-time setup), follow the steps in this chapter.

2.1.1 Download software

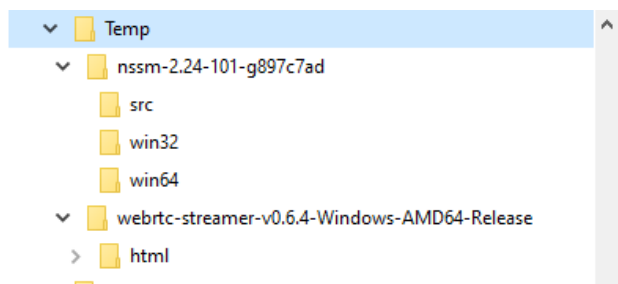
Download the public domain software components from their website as follows:

- webrtc-streamer: go to <https://github.com/mpromonet/webRTC-streamer/releases> and download the current version for Windows/AMD64 (the release version, if there are some to choose from). At time of writing, this is [webRTC-streamer-v0.6.4-Windows-AMD64-Release.tar.gz](https://github.com/mpromonet/webRTC-streamer/releases/download/v0.6.4-Release/webRTC-streamer-v0.6.4-Windows-AMD64-Release.tar.gz)
- nssm: go to <https://nssm.cc/download> and copy the “pre-release” version. At time of writing, this is [nssm-2.24-101-g897c7ad.zip](https://nssm.cc/download#nssm-2.24-101-g897c7ad.zip)

2.1.2 Unpack software

Unpack both software packages into a temporary folder of your choice. Note that you might need to use a 3rd party unpack application (like 7zip (see <https://www.7-zip.org/>)), as the Windows builtin unpack application cannot unpack *.tar.gz files.

After unpacking the software (to c:\temp here), you should get a folder structure as seen in the following screenshot:



The relevant executables of the software are now at:

- C:\Temp\nssm-2.24-101-g897c7ad\win64 (for nssm, only the single file nssm.exe is needed)
- C:\Temp\webRTC-streamer-v0.6.4-Windows-AMD64-Release (for webrtc streamer)

These folders are referenced as <nssm> and <webRTC-streamer> further on in this document.

NOTE: For production use, you will likely want to move nssm.exe to c:\windows\system32 and the <webRTC-streamer> folder to somewhere in c:\program files, e.g. c:\program files\webRTC-streamer.

2.2 Quick test

For a quick test, the webrtc-streamer software can be directly started by double-clicking the “webRTC-streamer.exe” from the <webRTC-streamer> folder. This will then start a webserver listening on <http://0.0.0.0:80>. The default installation also provides a webpage, which can be used to access a camera by specifying the cameras connection parameters as parameter for the webpage.

For an Axis camera, the live rtsp video feed is available using the following URL:

rtsp://<user>:<pass>@<ipaddr>/axis-media/media.amp

With parameters:

- <user> Username (default: root)
- <pass> Password (default: root)
- <ipaddr> IP address of the camera

Sample of a full Axis camera rtsp stream URL:

`rtsp://root:root@192.168.1.101/axis-media/media.amp`

To view this camera through the webrtc-streamer software, use the <http://localhost/webrtcstreamer.html> page and append the camera URL as `\video=` parameter. For the above given sample camera URL, this would then be as follows:

<http://localhost:8000/webrtcstreamer.html?video=rtsp://root:root@192.168.1.101/axis-media/media.amp>

Enter the URL in your web browser (for corporate PCs make sure to disable the proxy configuration in the webbrowser or disconnect the network cable). This will then show the live camera steam in the browser as follows:

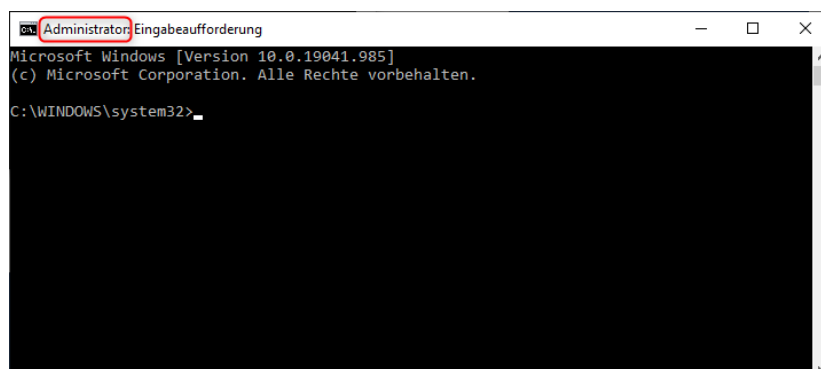


2.3 Service installation and configuration

To install the webrtc-streamer application as a windows service, the nssm software is used. The installation process also defines some configuration options for webrtc-streamer (through the command line), which makes the webrct-sreamer only accept requests from a browser running on the local machine (to prevent information leakage or unallowed remote access to the camera).

The nssm software is a command line application and must be started through a windows command line. Note that this requires administrative privileges!

To do so, click the Windows menu and type “cmd” – then richt click at “command prompt” app and select “run as administrator”. This will bring up a command window in an elevated context:



In the command window, enter the following command to start nssm in edit mode:

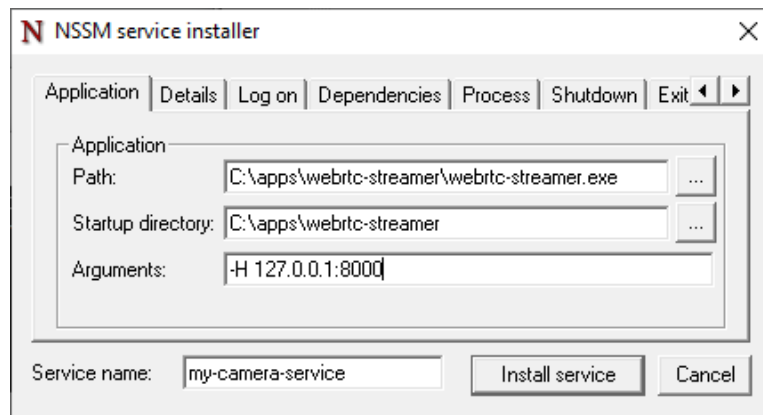
```
<nssm>\nssm.exe install <servicename>
```

where <nssm> is the path to your nssm installation folder and <servicename> is the name of the service to be created, e.g.

```
c:\windows\system32\nssm.exe install my-camera-service
```

This then brings up the service installation wizard.

On the first page (“application”), enter the following (replace c:\apps\webrtc-streamer with your <webrtc-streamer> installation folder):

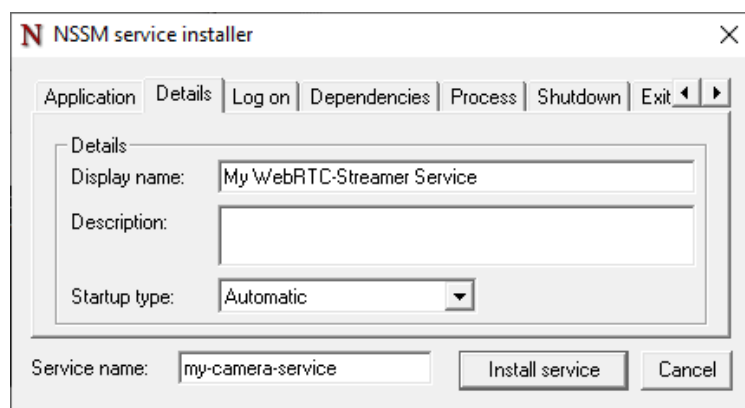


For the Arguments parameter, enter the following:

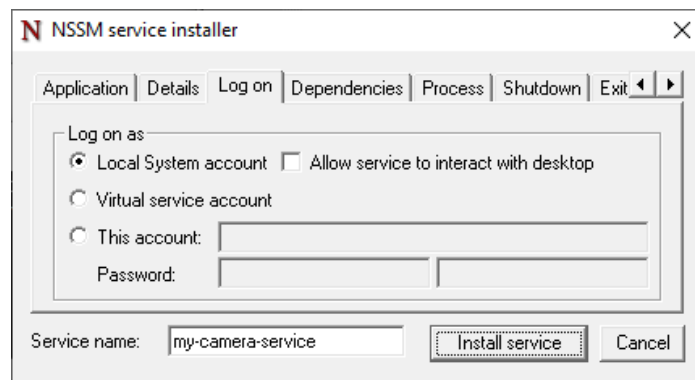
```
-H 127.0.0.1:8000
```

This makes the webrtc-streamer http server only listen to the localhost/loopback interface, so it won't accept any connections from the outside world. The number following the colon is the listening port number, by default this is :8000.

On the next page (“Details”), you can add more information. Leave the startup type at “Automatic” to start the service every time the system starts but set a meaningful display name (this is shown in the service control manager):

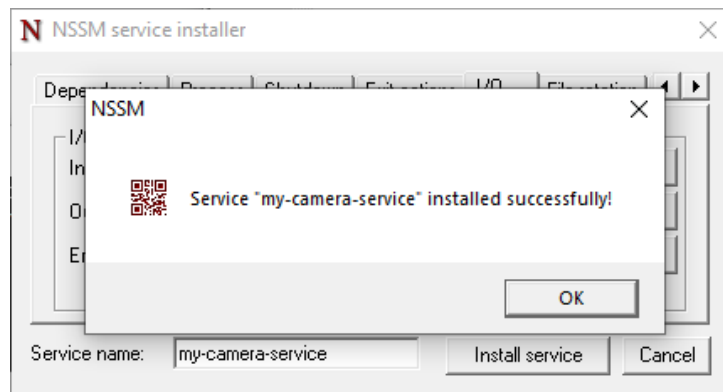


On the next page (“Log on”), leave everything as is (run the service in the LocalSystem account):

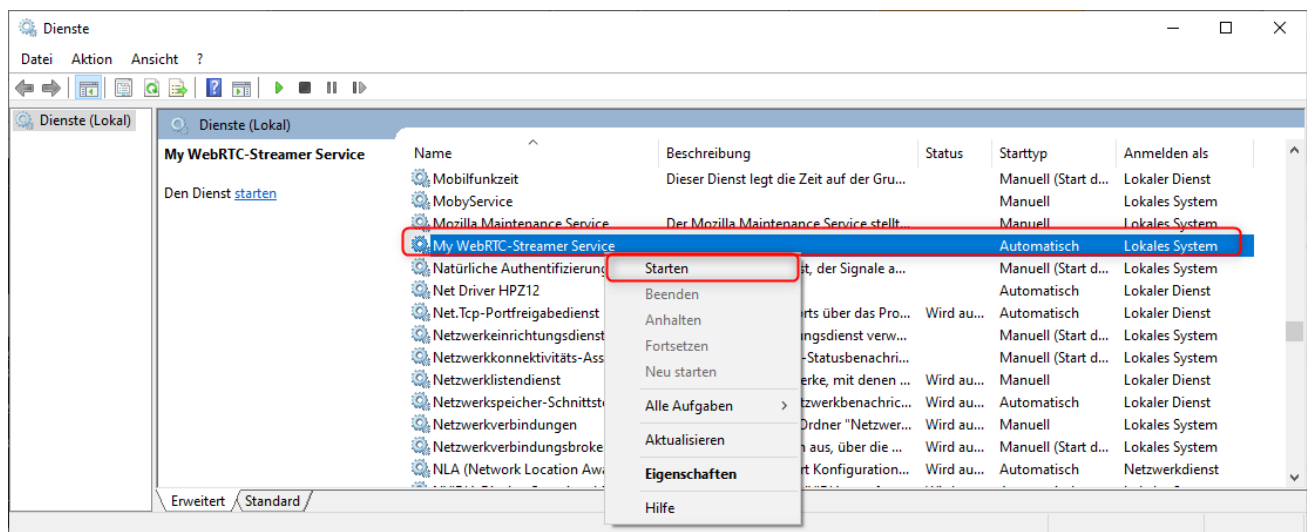


All other settings can be left as is.

To finish the service installation, click the “Install service” button. This will then register the service – if everything is ok, it will also show a confirmation:



To actually start the service, either reboot the system or open the Windows Service configuration manager. To do so, enter “services.msc” in the Windows start search box and hit enter. This will bring up the Services control panel applet, the new service can be found under whatever was given as the “Display Name” during installation:



To start the service, right click and select “start”.

If you get an error here, check, if the port number is not already allocated (or the test application from chapter 2.2 has been stopped!

2.4 Test

To test if the webrtc-streamer service is running, open the services control panel (see previous chapter) and check, if the state is set to “running”.

To test, if you can access the live video stream of the camera, open the same URL as shown in the “Quick test” in chapter 2.2.

NOTE: make sure to use the same base URL as configured in the -H parameter in the “Arguments” parameter in the service configuration (in the sample, http://127.0.0.1:8000).

To access the Axis camera as described in the quick test, open a browser and enter the following URL:

<http://127.0.0.1:8000/webrtcstreamer.html?video=rtsp://root:root@192.168.1.101/axis-media/media.amp>

This will then again show the live camera steam in the browser as follows:



2.5 Tweaking

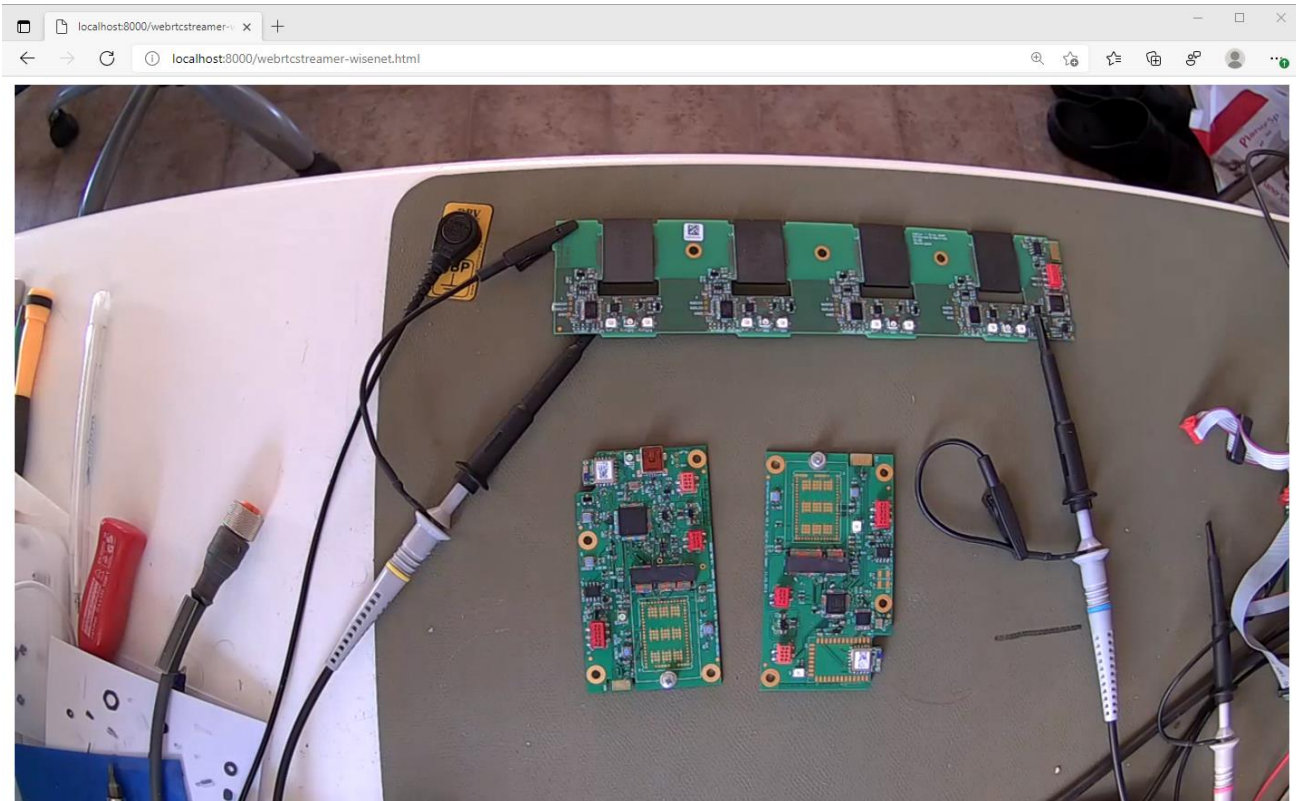
The webrtc-streamer allows placing custom html files into its html subfolder. These custom files are served through the webbrowser, so the actual view can be modified accordingly. Here is a sample html page source code, which includes the rtsp url and changes the view (removes header/footer texts):

```
<html>
<head>
<link rel="stylesheet" type="text/css" href="styles.css">
<style>body {margin:0;padding:0;background-color: black}</style>
<script src="libs/adaptter.min.js" ></script>
<script src="webrtcstreamer.json" ></script>
<script src="webrtcstreamer.js" ></script>
<script>
  var url = "rtsp://root:root@192.168.1.101/axis-media/media.amp";
  var options = webrtcConfig.options;
  window.onload = function() {
    this.webRtcServer = new WebRtcStreamer("video", webrtcConfig.url);
    webRtcServer.connect(url,null,options);
  }
  window.onbeforeunload = function() { this.webRtcServer.disconnect() }
</script>
</head>
<body>
  <div>
    <video style="background-color: #000;" width="100%" height="100%"
      id="video" muted playsinline controls></video>
  </div>
  <footer id="footer"></footer>
</body>
</html>
```

NOTE: you should change the cameras rtsp URL according to your camera!

Save this file into the <webrtc-streamer>\html folder as e. g. “axis.html”, then use the browser and connect to <http://127.0.0.1:8080/axis.html>.

This will then load the newly created file, which results in a clean stream web view – ready to integrate into OGS:

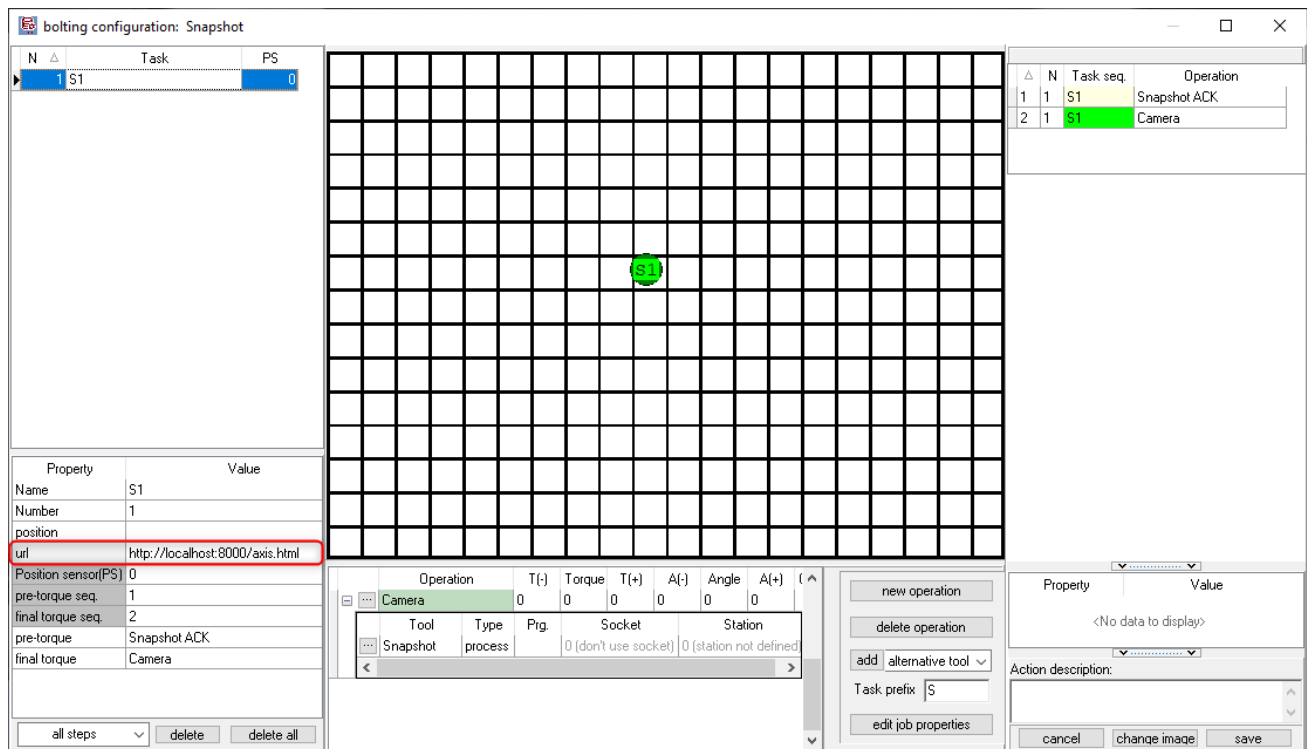


3 OGS integration

The final step to integrate a realtime camera stream into OGS is to create a task, which displays the web page in the OGS process stream.

3.1 Workflow configuration

Open the heOpCfg editor and open or create a job. The task “url” parameter is used to tell OGS to display a web page instead of the default – add the URL for the camera view (<http://127.0.0.1:8000/axis.html>) as created in the previous steps):



In the example shown this is actually assigned to a multistep task:

- The “pre-step” of the task is set to use the simple acknowledge tool (OK/NOK buttons shown on the screen).
- The “end-step” of the task is set to the camera capture tool.

The overall behaviour then is as follows:

- If the task gets active, the live video feed is shown (through web browser showing the webRTC-streamer url)
- The video feed is shown until the operator hits the “OK” button on the screen (inside the “pre-step” of the task)
- After the operator has clicked the OK, the “end-step” camera tool gets active and records the current video image and stores it as process result data.

After the saving the changes and exporting to the station, OGS will now show the live camera feed whenever the task gets active.

3.2 OGS Runtime

Whenever the task gets active, OGS will now show the live camera feed in the main browser area.

Here is a sample view:

